Analyzing a Decade of US Flight Delays

A Comparison of a Single Machine Approach (pandas) and a Cloud Big Data Approach (PySpark on AWS)

IST3134 Big Data Analytics

Group Assignment, May Semester 2026

Prepared by: Teo Zen Ben (22120679) and Jeanette Tan En Jie (23035173)

GitHub repository: https://github.com/ZenBen5173/IST3134-Flight-Delay-BigData

Dataset: US Bureau of Transportation Statistics, transtats.bts.gov

1. Introduction to the Problem

Flight delays are a common and costly problem in air travel and they affect passengers, airlines and airports at the same time. When a flight arrives late it can cause missed connections, extra crew and fuel costs and a domino effect, where the aircraft not only starts its own next trip late but also holds up other flights using the same runway, gates and crew, so understanding where and when delays happen is useful for planning and for setting expectations.

In this assignment we studied ten years of United States domestic flight records, from 2015 to 2024 and we asked which airports, routes, airlines and times of day had the worst delays and how the reasons behind those delays changed over the decade.

The dataset for a full decade is large, meaning it contains roughly 62 million flight records and this is deliberately too big to fit comfortably in the memory of a single laptop. This makes it a suitable problem for Big Data Analytics and it also lets us do the second and equally important part of the assignment, which is to compare two different ways of solving the same problem.

We built the same analysis twice, once using Python with the pandas library on a single machine, which is the ordinary non Big Data approach and once using Apache Spark (PySpark) running on an Amazon Web Services (AWS) cluster in the cloud, which is the Big Data approach. By running both at increasing data sizes and measuring how long each took and how much memory each used, we were able to show clearly why a distributed cloud tool is needed once the data grows large.

2. Introduction to the Dataset

The data comes from the Airline On-Time Performance collection published by the United States Bureau of Transportation Statistics (BTS), which is free to download and does not require any login. The download page is at https://www.transtats.bts.gov under the section “Reporting Carrier On-Time Performance (1987-present)”. Every domestic flight operated by the major US reporting airlines is recorded and each flight becomes one row in the data.

The data is published one file per month, so the full ten year period from 2015 to 2024 is made up of 120 monthly files, which together contain about 62 million flight records after cancelled flights are removed. Each row has around 110 columns and it describes one flight in detail, meaning it records which airline flew it, where it departed from and where it was going, the scheduled and the actual departure and arrival times, the distance, whether the flight was cancelled or diverted and, when a flight was late, a breakdown of the reason for the delay. This breakdown splits the lateness across five official causes, which are carrier (a problem the airline itself caused such as crew, maintenance or baggage), weather, national airspace system (air traffic control, heavy traffic and airport conditions), security and late arriving aircraft (a knock on delay from the same plane arriving late on its previous trip).

For our analysis we used a focused set of the columns rather than all 110, meaning we kept the airline, the origin and destination airports, the scheduled departure time, the arrival delay, the cancellation flag and the five delay cause columns. Throughout this report a flight is counted as delayed when its arrival

delay is 15 minutes or more, which is the standard definition used by the US Department of Transportation. Table 1 lists the main columns that we used and what each one means.

Column	Meaning
Reporting_Airline	The airline that operated the flight
Origin, Dest	Departure and destination airport codes
CRSDepTime	Scheduled departure time, used to derive the hour of day
ArrDelay	Arrival delay in minutes (15 or more counts as delayed)
Cancelled	1 if the flight was cancelled, otherwise 0
CarrierDelay, WeatherDelay, NASDelay, SecurityDelay, LateAircraftDelay	The five delay cause columns, in minutes

Table 1. The main columns used from the BTS On-Time Performance data.

3. Research Questions

The main question we set out to answer was, over the ten years from 2015 to 2024, which airports, routes, carriers and times of day had the worst flight delays and how the mix of delay causes shifted over the decade. We split this into five smaller questions so that each one maps cleanly to one result:

- **Q1 (Carrier).** Which airlines had the worst arrival delays, measured by the percentage of their flights that were delayed and their average arrival delay?
- **Q2 (Airport).** Which origin airports had the worst departure performance?
- **Q3 (Route).** Which origin to destination routes had the worst delays?
- **Q4 (Time of day).** How does the chance of a delay change with the scheduled departure hour and which hours are the worst to fly?
- **Q5 (Cause mix over time).** How has the share of total delay coming from each of the five causes changed from year to year across the decade?

4. Scope and Limitations

This project covers United States domestic flights recorded in the BTS On-Time Performance data over the ten years from 2015 to 2024 and within that data we look only at the five questions set out above, which are delays by carrier, airport, route and time of day together with the yearly mix of delay causes. A flight is treated as delayed when its arrival delay is 15 minutes or more. The Big Data goal is a fair comparison of two ways of running the same group by aggregation, so our focus is on how the runtime and the memory behave as the data grows rather than on building the fastest possible pipeline.

Several things are deliberately left outside our scope. We do not try to predict or model delays, so there is no machine learning, because the aim was the aggregation itself and the comparison of the two tools. We do not look at international flights, airfares, passenger numbers or the wider cost of delays. Cancelled flights are simply removed rather than studied on their own. We also do not tune either system for maximum speed and we did not resize the cluster or change the Spark settings, so the runtimes should be read as a like for like comparison on a standard setup rather than the best each tool could reach. Finally the cloud upload was slowed by the data source limiting our requests, which we note where it is relevant.

5. The Algorithm and How Spark Processes the Data

The algorithm we used is a group by aggregation, which is the same idea as the classic MapReduce pattern that Big Data platforms are built around. We developed the aggregation logic ourselves with assistance from AI tools and we wrote it using the ready made grouping functions that pandas and Spark already provide. The documentation for both libraries is listed in the references at the end of the report. In plain terms the algorithm answers each question by sorting the flights into groups, for example one group per airline and then summarizing each group.

Behind the scenes, Spark breaks the work into three stages. In the map stage each flight record is read once and turned into a key and a small set of values, for example the key is the airline and the values are a count of one, the arrival delay in minutes and a 1 or 0 flag for whether the flight was delayed. In the shuffle stage the platform gathers together all the values that share the same key, meaning all the records for the same airline are brought to the same place. In the reduce stage the values in each group are combined, meaning the counts are added up, the delay minutes are averaged and the delayed flags are averaged to give the percentage of delayed flights. The same three stage pattern is used for the airport, route, hour and yearly cause groupings, only the key changes.

The reason this suits Big Data is the way Spark carries out the work. pandas loads the entire dataset into the memory of one machine and then processes it, so the memory needed grows with the size of the data. Spark instead splits the data into many partitions, meaning smaller chunks and spreads them across the machines in a cluster so that many parts of the map and reduce work happen at the same time. When memory runs low Spark writes intermediate data to disk (this is called spilling) instead of failing, so it can process data that is far larger than the memory of any single machine. This is why the same algorithm scales to the full decade on Spark while it strains the single machine version and it is the central point that this assignment is designed to demonstrate.

6. Implementation and Experimental Setup

We implemented the same analysis in two ways so that we could compare a non Big Data approach against a Big Data approach on identical work.

6.1 The single machine baseline (pandas)

The baseline was written in Python using the pandas library and it was run on one laptop. It reads the monthly files, removes cancelled flights, creates the delayed flag and the departure hour and then runs the five group by aggregations. This version keeps all of the data in memory, so it is simple and fast for small inputs but it is expected to strain as the number of years grows.

6.2 The Big Data approach (PySpark on AWS)

The Big Data version was written in PySpark, which is the Python interface to Apache Spark and it was run on a cluster on Amazon Web Services. We uploaded the data to Amazon S3, meaning cloud storage and we then created an Amazon EMR cluster, which is a managed Spark and Hadoop platform, made up of one primary node and two core nodes.

The Spark job reads the data directly from S3, runs the exact same five aggregations and writes the results back to S3. The cluster ran Spark 4.0.2 on the emr-spark-8.0.0 release and the screenshots of the running cluster, the completed job and the success log are included with our submission as evidence that the analysis was carried out in the cloud.

An important check is that the two implementations agree, because they run the same algorithm on the same data they should give the same answers and they did, meaning the result tables from pandas and from Spark matched to the decimal, for example American Airlines showed the same 29.29% of flights delayed for January 2024 in both. This shows that the comparison between the two is fair, because any difference in speed or memory comes from the tools and not from the calculation.

To measure how each approach scales we ran both of them over four increasing data sizes, which were one month (January 2024), one year (2024), five years (2020 to 2024) and ten years (2015 to 2024) and for each run we recorded the number of rows, the total runtime and, for pandas, the peak memory used.

7. Analysis of the Output

This section presents the findings for the full ten year period. Each result is shown as a chart together with a short table so that the numbers stated in the text can also be seen in the figures.

7.1 Q1 Worst airlines

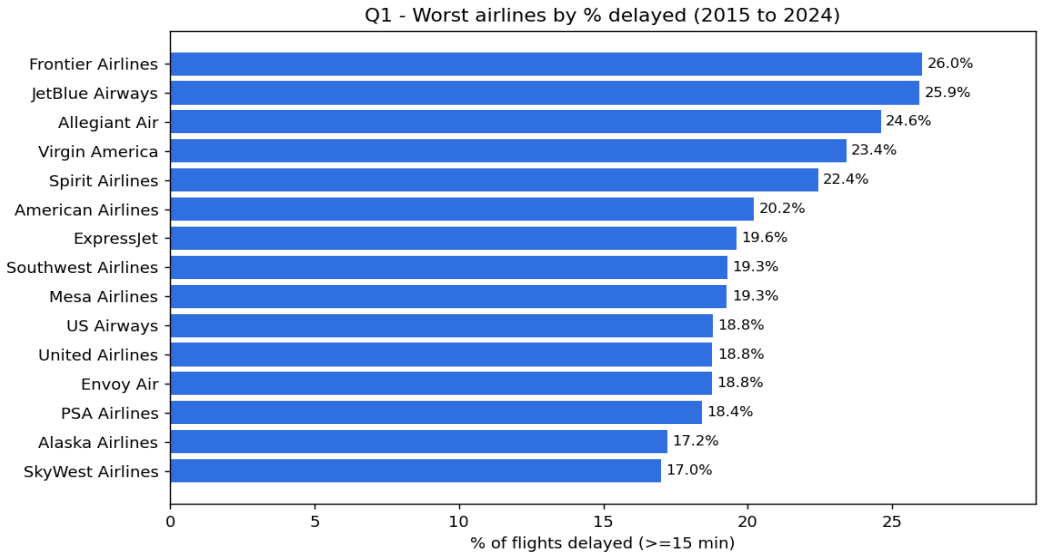

Figure 1. Airlines ranked by the percentage of flights delayed, 2015 to 2024.

The airlines with the worst delay records over the decade were the low cost carriers. Frontier Airlines was the worst with 26.0% of its flights delayed, followed by JetBlue at 25.9%, Allegiant at 24.6%, Virgin America at 23.4% and Spirit at 22.4%, as shown in Figure 1 and Table 2. The pattern in the chart is that the top of the ranking is filled almost entirely by budget carriers while the large network carriers such as Delta, Alaska and Hawaiian sit at the reliable end. A likely reason is the budget model itself, because these airlines keep their aircraft flying as much as possible with short turnarounds and little spare time in the schedule, so there is very little slack to absorb a delay and any early disruption is passed on to later flights. This link

between scheduled slack and delay is well established, where re-allocating buffer time in a schedule has been shown to measurably reduce how far delays propagate (AhmadBeygi, Cohn, & Lapp, 2010).

Airline	Flights	Avg arrival delay (min)	% delayed
Frontier (F9)	1,287,500	11.3	26.0
JetBlue (B6)	2,528,111	10.9	25.9
Allegiant (G4)	739,094	11.8	24.6
Virgin America (VX)	217,006	7.4	23.4
Spirit (NK)	1,837,073	8.0	22.4

Table 2. The five airlines with the highest percentage of delayed flights.

7.2 Q2 Worst airports

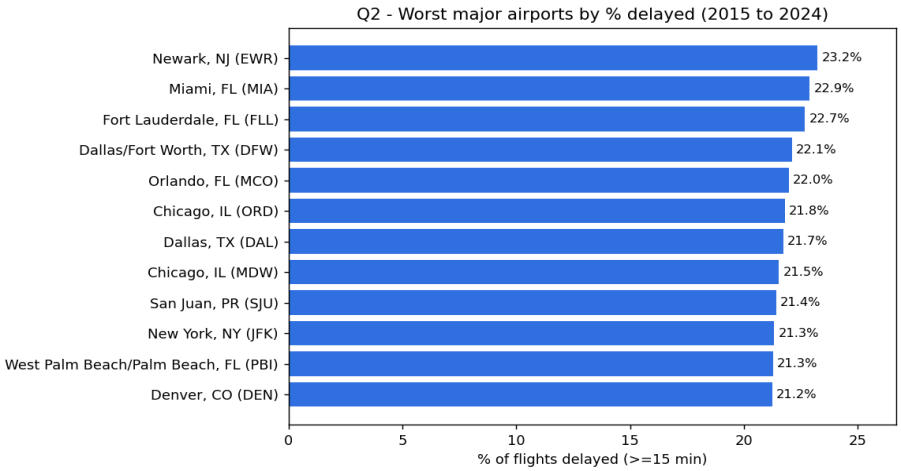

Figure 2. Worst major origin airports (at least 100,000 flights over the decade).

Among the busy airports, those with at least 100,000 departures over the decade so that the result is not distorted by tiny low traffic airports, Newark was the worst at 23.2% of flights delayed, followed by Miami at 22.9%, Fort Lauderdale at 22.7%, Dallas Fort Worth at 22.1% and Chicago O'Hare at 21.8%, as shown in Figure 2. The more telling pattern in the chart is that the worst airports are not scattered across the country, they cluster by city. Dallas appears twice with Fort Worth and Love Field, Chicago appears twice with O'Hare and Midway and four of the twelve are Florida airports. When both airports serving the same city land near the top, the delays look like a shared regional problem rather than one badly run airport. This matches the FAA idea of a metroplex, a metropolitan area where a large number of airports are concentrated and the air traffic becomes congested, and several of our worst airports including the Dallas Fort Worth area, the Florida airports and Denver fall in the exact metros the FAA singled out for congestion relief (U.S. Department of Transportation, Office of Inspector General, 2019).

7.3 Q3 Worst routes

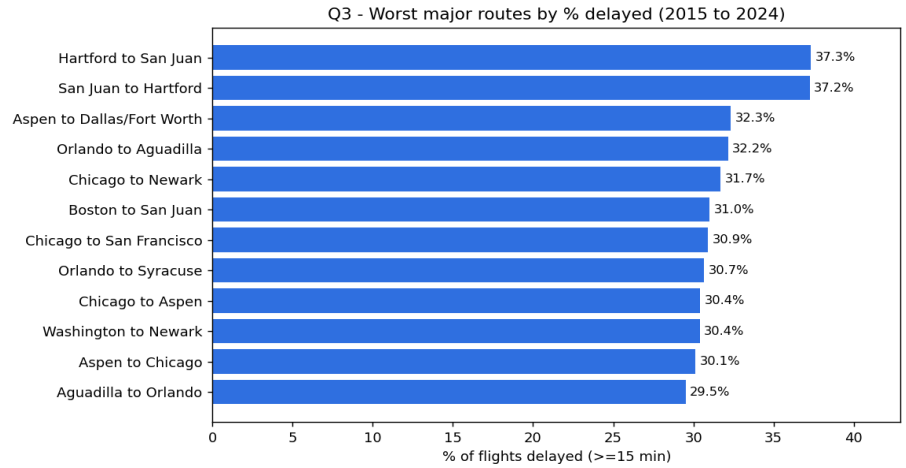

Figure 3. Worst major routes (at least 5,000 flights over the decade).

Looking at specific origin to destination routes with at least 5,000 flights over the decade, the worst were the connections between Hartford and San Juan in Puerto Rico, where about 37% of flights were delayed in each direction, followed by Aspen to Dallas Fort Worth at 32.3% and several routes out of Chicago Midway and Newark, as shown in Figure 3. The clearest pattern in the chart is that the worst routes appear as mirrored pairs, so Hartford to San Juan and San Juan to Hartford both sit at the top, as do Orlando to Aguadilla and Aguadilla to Orlando. This points to delay propagation on a single aircraft rotation, because the same plane usually flies out and straight back on these thin routes, so a late inbound leg forces the return leg to leave late as well. This connection driven propagation through shared aircraft is exactly what has been shown to spread delay across an airline network (AhmadBeygi, Cohn, Guan, & Belobaba, 2008). The routes are also mostly long leisure markets such as Puerto Rico and Aspen, which fits the airport results because Newark and the Florida and Puerto Rico markets appear again.

7.4 Q4 Delay by time of day

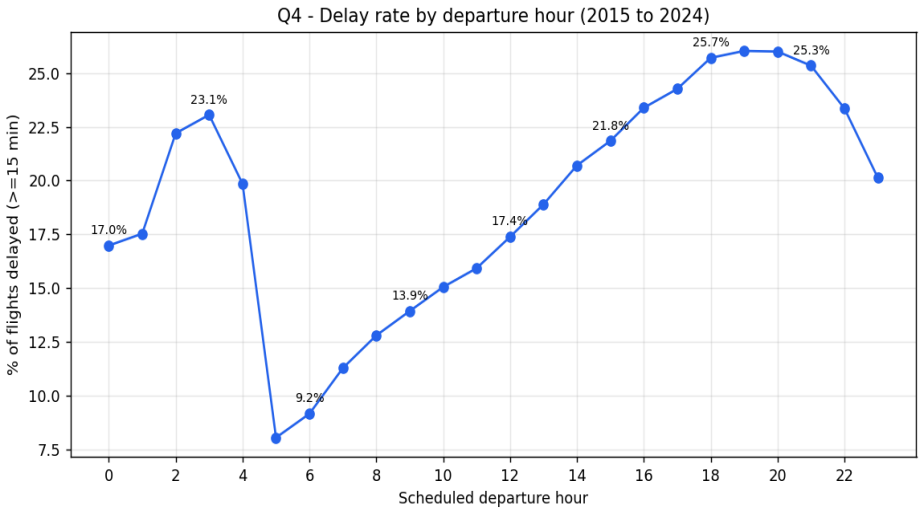

Figure 4. Percentage of flights delayed by scheduled departure hour, 2015 to 2024.

The time of day has a strong and very consistent effect. Early morning flights were the most reliable, so flights scheduled to leave between 5am and 7am actually arrived a few minutes early on average and only about 8% to 11% were delayed, and the delay rate then built up steadily through the day to a peak in the early evening, where flights leaving around 6pm to 7pm had the highest average delay and about 26% of them were delayed, as shown in Figure 4 and Table 3. The shape of this curve is the knock on effect made visible, because small delays early in the day accumulate as the same aircraft and crews are reused later. This is supported by how the delay is officially recorded, since the largest single reported cause is late arriving aircraft, which BTS defines as a previous flight with the same aircraft arriving late and causing the next one to leave late (Bureau of Transportation Statistics, n.d.-b). The practical advice that follows is simple, an early flight is much more likely to be on time than a late afternoon or evening one.

Scheduled departure hour	Avg arrival delay (min)	% delayed
06:00	-2.7	9.2
09:00	0.1	13.9
12:00	3.5	17.4
15:00	7.5	21.8
18:00 (worst)	10.8	25.7
21:00	10.0	25.3

Table 3. Selected departure hours showing how delay rises through the day.

7.5 Q5 How the delay causes changed over the decade

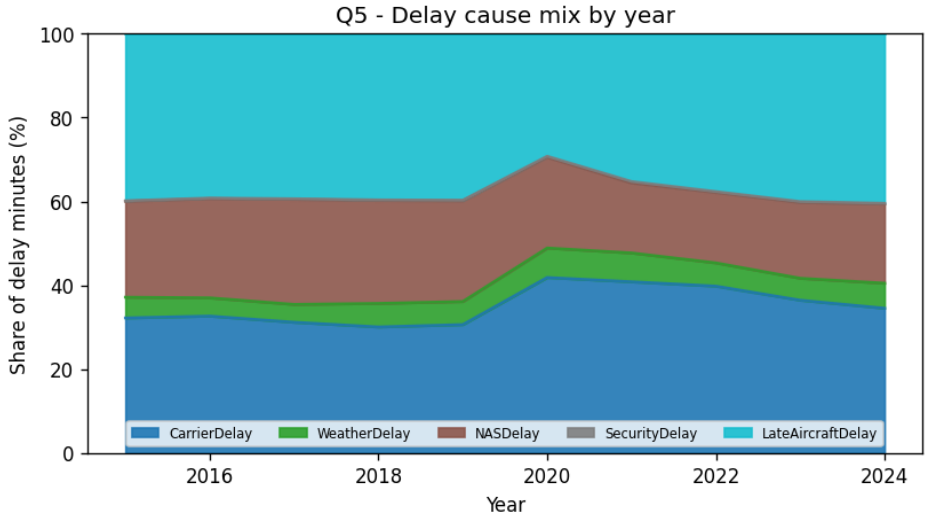

Figure 5. Share of total delay minutes by cause, each year from 2015 to 2024.

Across the whole decade the two largest causes of delay were late arriving aircraft and the carrier itself, which together made up roughly 70% of all delay minutes, while weather stayed small at around 4% to 7% and security was almost negligible at well under half a percent, as shown in Figure 5 and Table 4. The pattern is that delay is mostly an internal operational problem rather than an external one, because the two dominant categories both sit inside the airline and airspace system rather than being caused by weather or security. It is worth noting that this small weather share understates the true role of weather,

because BTS records non extreme weather inside the airspace and late aircraft categories rather than as its own line (Bureau of Transportation Statistics, n.d.-b), which is one reason we did not try to attribute the airport pattern to weather. The clearest change happened in 2020, the first year of the COVID pandemic, when carrier caused delay jumped to 41.8% of the total and late aircraft delay fell to 29.2%, which fits the idea that with far fewer flights in the schedule there were fewer aircraft to pass delays down the chain. As flying returned to normal the mix moved back and by 2024 late aircraft was again the largest cause at 40.4%.

Year	Carrier %	Weather %	NAS %	Security %	Late aircraft %
2015	32.2	4.9	22.9	0.1	39.8
2019	30.6	5.5	24.0	0.1	39.7
2020	41.8	7.0	21.7	0.2	29.2
2022	39.8	5.6	16.8	0.2	37.6
2024	34.5	6.0	18.9	0.2	40.4

Table 4. Delay cause mix for selected years, as a share of total delay minutes.

8. Performance Comparison: Single Machine versus Cloud Big Data

The results above answer the flight delay question and they are the same no matter which tool produced them. The purpose of this section is different, meaning it compares how the two implementations behaved as the data grew, which is the core Big Data argument of the assignment. Table 5 lists the measured runtime for both approaches and the peak memory for the pandas baseline at each of the four data sizes.

Data size	Rows	pandas runtime	pandas peak memory	Spark on AWS runtime
1 month	526,882	8 s	under 1 GB	23.7 s
1 year	6,982,746	41.3 s	4.33 GB	33.7 s
5 years	30,590,198	227.4 s	19.05 GB	126.7 s
10 years	61,839,886	479.0 s	38.44 GB	250.3 s

Table 5. Measured runtime and memory for both approaches at each data size.

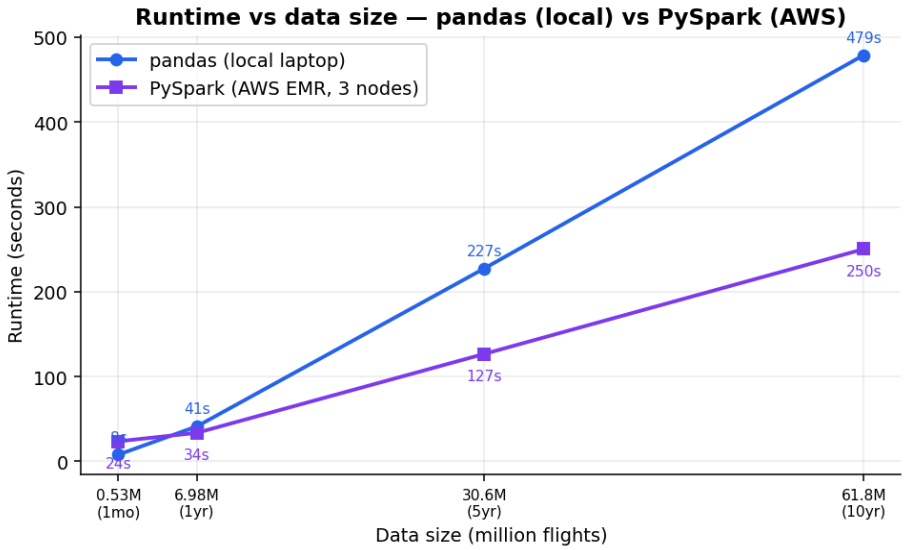

Figure 6. Runtime against data size for pandas (local) and PySpark (AWS).

The runtime comparison in Figure 6 shows a clear crossover. For the smallest data, one month, pandas was actually faster at 8 seconds against 23.7 seconds for Spark, because Spark has to start up a cluster, read from S3 and coordinate work across machines and this fixed overhead is not worth it for a tiny input. The two approaches met at around one year and beyond that Spark pulled ahead, so at ten years Spark finished in 250.3 seconds while pandas took 479.0 seconds, which is about 1.9 times faster. This is the expected behaviour of a distributed framework, which carries a fixed cost to split the work, move the data and coordinate the machines and therefore only pays off once the data is large enough for that parallel work to outweigh the startup cost (Dean & Ghemawat, 2008).

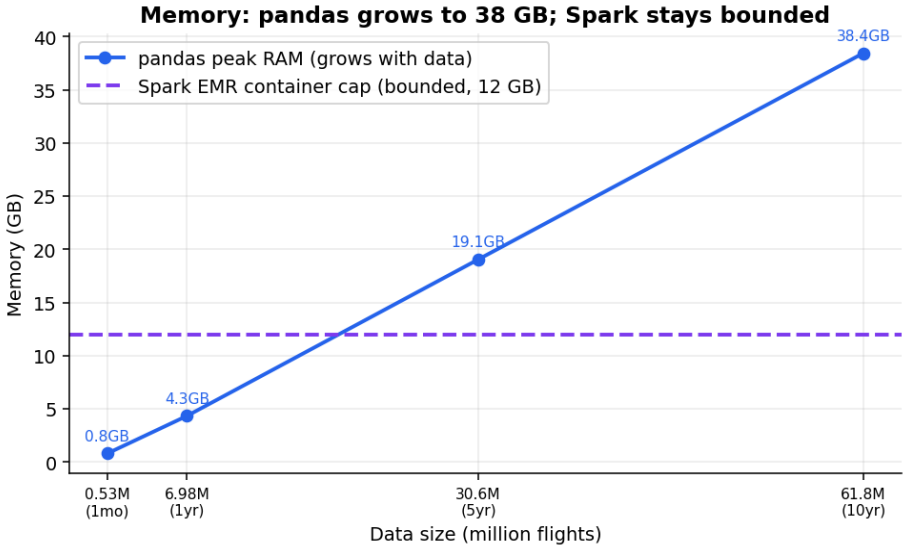

Figure 7. Peak memory: pandas grows with data size while Spark stays bounded.

The memory comparison in Figure 7 is even more important than the runtime. Because pandas holds everything in memory, its peak memory grew almost in a straight line with the data, from under 1 GB for one month up to 38.44 GB for ten years. Our laptop happened to have enough memory to finish, but a

more typical laptop with 8 GB or 16 GB would have run out of memory and crashed at the five or ten year sizes. Spark kept its memory bounded because it processes the data in partitions and spills to disk when needed rather than loading everything at once, so it handled the full decade within the fixed container size of about 12 GB on the cluster (Zaharia, Chowdhury, Franklin, Shenker, & Stoica, 2010). This is the real limitation of the single machine approach, which is not only speed but that it simply cannot hold very large data, and it is exactly the limitation that a Big Data platform is designed to remove.

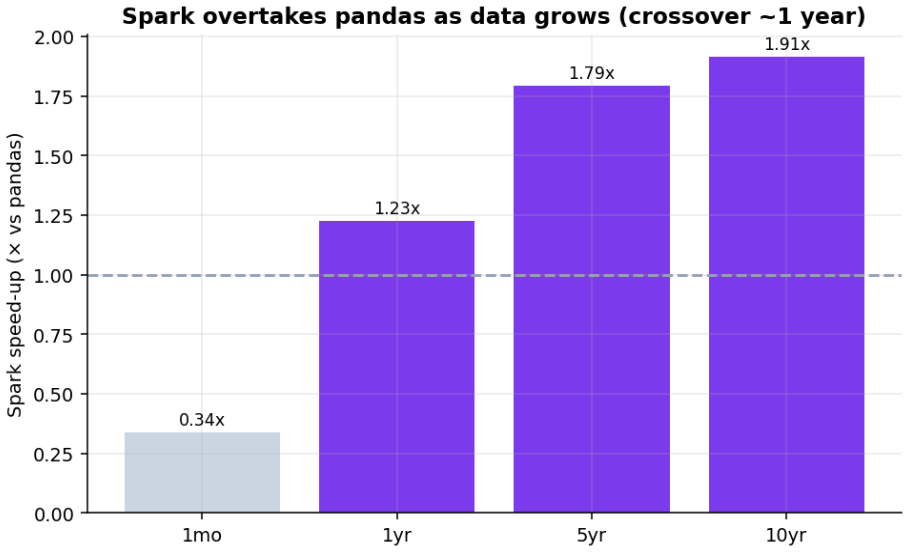

Figure 8. How many times faster Spark is than pandas at each data size.

Figure 8 summarizes the speed up, the pandas runtime divided by the Spark runtime, and it makes the trend easy to see, since Spark starts below 1 and is slower than pandas at one month then rises to about 1.9 times faster at ten years. The speed up is real but modest, which is expected, because the overall gain from parallel processing is always held back by the parts of the job that do not parallelise and by the fixed coordination overhead (Amdahl, 1967). The honest conclusion is that a single machine tool such as pandas is a good and simple choice for small data, but as the data grows it becomes slower and, more seriously, its memory need grows until the machine can no longer cope, whereas the distributed Spark approach on AWS scales out across a cluster, keeps its memory bounded and completes the full ten years comfortably.

9. Individual Reflections

Each group member has written their own reflection on what they learned from this assignment.

9.1 Reflection: Teo Zen Ben (22120679)

For this assignment I focused mainly on the technical side, meaning I set up and ran both the pandas baseline and the Spark version and I handled the work on Amazon Web Services. The most useful thing I learned is that a Big Data tool is not automatically better, because for one month of data pandas actually finished faster than Spark, since Spark first spends time starting up a cluster and reading from S3 before it does any real work. It was only when the data grew to five and ten years that Spark pulled ahead and, more importantly, kept its memory bounded while pandas climbed to over 38 GB and seeing those two

lines cross in our own results made the idea of scaling much clearer than just reading about it. I also learned that a lot of Big Data work is really about getting the data and the platform set up correctly rather than the analysis itself, because I ran into real problems such as the data source rate limiting our downloads and the EMR cluster failing the first time because the service role did not have the right permissions and fixing these taught me how S3, EMR and the IAM roles fit together. Overall I now understand why this kind of work is moved to a cluster in the cloud, since a single machine simply cannot hold the full ten years in memory while the cluster handled it comfortably.

9.2 Reflection: Jeanette Tan En Jie (23035173)

My main role was the writing and the report, meaning I took the results that came out of the code and turned them into a clear explanation that answers our question. I learned that having the numbers is only half of the work, because the harder part is deciding what each result actually means and saying it plainly, for example explaining why delays build up through the day as the same aircraft is reused, or why carrier caused delay rose sharply in 2020 when there were far fewer flights to pass delays down the chain. To write the Big Data section properly I had to understand the map and reduce idea and how Spark spreads the work across a cluster well enough to put it into everyday language and having to explain it for the report is what made me actually understand it. I also learned to be honest about the limitations, for example that our analysis counts a flight as delayed only when it arrives fifteen or more minutes late and that it describes the patterns in the data rather than proving their exact causes and that stating this clearly is better than hiding it. Working closely with the technical side helped me connect each chart and number back to the original question and I now feel much more confident reading and explaining this kind of performance comparison.

10. Conclusion

Over ten years of United States flight data we found that the low cost airlines had the worst delay records, that the most delay prone busy airports were congested hubs such as Newark, Miami and Chicago while Hawaii airports were the most reliable, that delays build up steadily through the day so that early morning flights are far more reliable than evening ones and that late arriving aircraft and carrier problems are the two biggest causes of delay, with a clear shift toward carrier caused delay during the pandemic year of 2020. Beyond these findings, the comparison between the two implementations showed why Big Data tools matter, because the single machine pandas approach was simple and even faster on small data but its memory grew to over 38 GB on the full decade, while the same analysis on Apache Spark running on an AWS cluster kept its memory bounded and ran the full ten years faster, which demonstrates the advantage of a distributed Big Data platform in the cloud once the data becomes large.

11. Declaration on the Use of AI

We used an AI assistant (Claude) as a support tool during this assignment and we declare its use here for full transparency. We first used it to speed up the planning stage, because it helped us search and shortlist many possible datasets across different domains until we settled on this one, which is far larger than a typical group dataset of around a million rows and therefore lets us show the single machine versus Big

Data comparison much more clearly. We then used it to help us write the pandas code and to diagnose errors as they came up, especially during the AWS setup. It also helped us lay out the structure of this report, check the finished report against the marking rubric several times and check our grammar. Separately, the airline codes in the dataset such as F9 or B6 were converted to their full names such as Frontier Airlines and JetBlue Airways with the help of the AI, because the data files only contain the codes and not the names.

All of the results, tables and charts come from the real data and from code that we ran ourselves. The AI did not create or change any numbers and we confirmed that the pandas and Spark outputs matched before relying on them. The interpretation of the findings, the final decisions and the individual reflections are our own.

12. References

AhmadBeygi, S., Cohn, A., Guan, Y., & Belobaba, P. (2008). Analysis of the potential for delay propagation in passenger airline networks. Journal of Air Transport Management, 14(5), 221–236.

AhmadBeygi, S., Cohn, A., & Lapp, M. (2010). Decreasing airline delay propagation by re-allocating scheduled slack. IIE Transactions, 42(7), 478–489.

Amazon Web Services. (n.d.). Amazon EMR documentation. https://docs.aws.amazon.com/emr

Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. In Proceedings of the April 18–20, 1967, Spring Joint Computer Conference (pp. 483–485). Association for Computing Machinery.

Apache Software Foundation. (n.d.). Spark SQL, DataFrames and Datasets guide. https://spark.apache.org/docs/latest

Bureau of Transportation Statistics. (n.d.-a). Reporting Carrier On-Time Performance (1987–present) [Data set]. United States Department of Transportation. https://www.transtats.bts.gov

Bureau of Transportation Statistics. (n.d.-b). Understanding the reporting of causes of flight delays and cancellations. United States Department of Transportation. https://www.bts.gov/topics/airlines-and-airports/understanding-reporting-causes-flight-delays-and-cancellations

Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. Communications of the ACM, 51(1), 107–113.

The pandas development team. (n.d.). pandas documentation: Group by (split-apply-combine). https://pandas.pydata.org/docs

U.S. Department of Transportation, Office of Inspector General. (2019). FAA has made progress in implementing its Metroplex program, but benefits for airspace users have fallen short of expectations (Report No. AV2019062). https://www.oig.dot.gov

Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster computing with working sets. In Proceedings of the 2nd USENIX Workshop on Hot Topics in Cloud Computing (HotCloud '10). USENIX Association.